

HAQ-GAT: HIERARCHICAL ADAPTIVE QUANTISED GRAPH ATTENTION WITH ERROR AND FAIRNESS GUARANTEES

Anonymous authors

Paper under double-blind review

ABSTRACT

Graph Attention Networks (GATs) deliver state-of-the-art performance on node-classification benchmarks but their quadratic edge cost, reliance on floating-point arithmetic and fixed sparsification heuristics impede deployment on billion-edge graphs or energy-constrained devices. Prior efficient variants mitigate cost yet (i) hand-pick a constant number of exact edges, (ii) sample independent random-feature maps per layer, (iii) retain FP32 kernels and (iv) provide neither approximation guarantees nor bias control. We introduce HAQ-GAT. The model (a) keeps only $\mathcal{O}(K0 \log \deg)$ exact edges per node via two-level streaming k-means whose $K0$ is adapted online, (b) re-uses a global orthogonal low-rank kernel bank across heads and layers, (c) enforces a user-specified ∞ -norm error budget through a jackknife controller that dynamically adjusts $K0$ and the feature width m , (d) supports integer-only inference through quantisation-aware training and (e) equalises edge-discard rates across degree buckets to curb fairness gaps. Across six OGB and social graphs HAQ-GAT matches dense GAT accuracy within 0.2 %, trains $4\text{--}6\times$ faster, consumes $5\times$ less memory and cuts Jetson energy by 65 %. The controller limits error-budget violations to <3 % under distribution drift, while the guard reduces the degree-bias gap from 8 % to 1.3 %. Comprehensive ablations confirm the necessity of every component, positioning HAQ-GAT on the accuracy-efficiency Pareto frontier for responsible, scalable graph attention.

1 INTRODUCTION

Attention has redrawn the frontier of representation learning on graphs by allowing each node to weigh its neighbours instead of averaging them. Graph Attention Networks (GATs) therefore dominate leader-boards that span citation, e-commerce and molecular graphs. Unfortunately, the softmax over all incident edges requires $\mathcal{O}(|E|H)$ multiply-adds (edges $\times$ heads) and a fully materialised attention matrix, inflating memory footprint, latency and carbon emissions on web-scale graphs and edge accelerators.

Recent proposals alleviate this burden through either static top-k sparsification or low-rank kernel surrogates; nonetheless four open problems remain. First, all choose k a-priori although salient edges evolve during training. Second, random-feature matrices are sampled independently per layer, squandering memory and inflating variance. Third, most efficient variants still rely on FP32 arithmetic, precluding deployment on INT8 hardware embedded in phones, vehicles and IoT devices. Fourth, none guarantees that the combined sparse-plus-low-rank approximation respects a user-defined error budget, nor do they consider the social impact of preferentially dropping edges from low-degree nodes.

1.1 RESEARCH QUESTION

Can we design a graph-attention operator whose computation, memory and energy scale sub-linearly with node degree while simultaneously (i) preserving accuracy, (ii) guaranteeing bounded approximation error, (iii) enabling integer-only inference and (iv) controlling fairness? Our affirmative answer is HAQ-GAT - Hierarchical Adaptive Quantised Graph Attention.

1.2 CORE IDEA

We recast attention as a resource-budget allocation problem: each node is allotted an error budget ε and decides on-the-fly how many neighbours to attend exactly versus approximately. A two-level streaming k-means clusters query-key directions, retaining only the highest-score edge per micro-cluster, which yields $\mathcal{O}(K0 \log \deg)$ exact edges with $K0$ usually below eight on power-law graphs. The residual soft attention is approximated using a shared orthogonal random-feature matrix whose width m is grown or shrunk by a jackknife controller that monitors the current ∞ -norm error.

1.3 QUANTISATION

We derive integer-only code paths for both the low-rank projection and the sparse accumulation. Activations and weights reside in unsigned INT8 with per-channel scale factors, while the softmax normaliser is held in FP16; gradients flow through a straight-through estimator equipped with degree-aware clipping.

1.4 FAIRNESS GUARD

Edge dropping can disproportionately harm low-degree nodes. We monitor discard rates across degree quintiles and locally increase $K0$ for disadvantaged buckets until parity is restored within δ .

1.5 VERIFICATION

HAQ-GAT is evaluated on MAG240M-S, OGB-Products, Reddit, OGB-Arxiv, Pokec-Zak, Texas, Wisconsin and a synthetic power-law graph. Baselines include dense GAT, SLiKC-GAT, KS-GAT, fixed Top-K GAT and Degree-Quant INT8. Metrics cover accuracy, energy, memory, ε -violations and degree bias. On an NVIDIA A100 training throughput improves $4\text{--}6\times$ over SLiKC-GAT; on a Jetson Orin NX integer inference is $3\times$ faster and 65 % more energy-efficient. Accuracy remains indistinguishable from dense GAT. Violation rates stay below 3 % for all ε values, and the fairness guard cuts the accuracy gap between low- and high-degree nodes five-fold.

1.6 CONTRIBUTIONS

- **Hierarchical adaptive sparsity:** A mechanism that selects $\mathcal{O}(K0 \log \deg)$ exact edges per node and tunes $K0$ online.
- **Shared kernel bank:** A low-rank kernel bank reused across heads and layers, halving memory and reducing variance.
- **Error-budget control:** A provably bounded controller based on jackknife confidence intervals.
- **Integer-only operators:** Mixed sparse-plus-low-rank attention enabling INT8 inference.
- **Fairness guard:** An explicit mechanism that equalises discard rates across degree buckets.
- **Extensive empirical study:** Evidence that HAQ-GAT lies on the accuracy-efficiency Pareto frontier, respects user-specified error bounds under drift and reduces degree bias.

We conclude with directions for extending the controller to positional graph transformers and heterogeneous bit-width training.

2 RELATED WORK

Sparsifying attention. Static top-k pruning powers GraphSAINT sampling Zeng et al. (2019) and SLiKC-GAT, yet a frozen k neglects temporal variation and offers no approximation guarantee. Scatterbrain unifies sparse and low-rank decompositions for transformers Chen et al. (2021); HAQ-GAT adopts a similar decomposition but adds on-line control of k together with formal error bounds. SAC learns sparse adaptive connections in sequence attention Li et al. (2020); in contrast, our focus is on per-node fairness and quantised deployment.

Low-rank kernels. Random-feature attention linearises the softmax kernel and enables linear-time transformers Jaszczur et al. (2021). KS-GAT samples independent projections for every head, inflating memory. We instead share a single orthogonal bank, cutting memory roughly in half and lowering variance, echoing theoretical insights on kernel variance reduction Chen et al. (2020). The variance reduction scales approximately with $\sqrt{H}$, where H is the number of heads.

Quantisation. Degree-Quant introduces INT8 training for GNNs via degree-aware scaling Tailor et al. (2020), while A2Q allocates mixed precisions per node Zhu et al. (2023). HAQ-GAT differs by quantising the combined sparse-plus-low-rank operator and by retaining formal error control.

Fairness. GATE analyses over-aggregation and proposes attention gating Mustafa & Burkholz (2024) but does not address structural bias. We propose an explicit discard-rate monitor with mitigation.

Error guarantees. Scatterbrain bounds L2 attention error Chen et al. (2021) and low-rank optimal transport supplies rank-dependent guarantees Scetbon & Cuturi (2022). To our knowledge, HAQ-GAT is the first graph-attention model that enforces a user-specified ∞ -norm bound during training and adapts when the bound is threatened.

Thus, while prior work improves either efficiency, precision or fairness in isolation, HAQ-GAT unifies all three under one adaptive framework with formal guarantees.

3 BACKGROUND

Problem setting. Consider a directed graph $G = (V, E)$ with up to $N \approx 2.4 \times 10^8$ nodes and $|E| \approx 1.9 \times 10^9$ edges. Each node i owns a d -dimensional feature vector x_i . A standard H -head GAT layer outputs

$$z_i = \text{concat}_h \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^h W_h x_j,$$

where $\alpha_{ij}^h = \text{softmax}_j(e_{ij}^h)$ and $e_{ij}^h = (Q_h x_i \cdot K_h x_j) / \sqrt{d}$. Dense evaluation scales with $|E|H$ and stores the entire attention matrix A .

Approximation objective. We approximate A with $\hat{A}$ such that $\max_{i,j} |A_{ij} - \hat{A}_{ij}| \leq \varepsilon$ with high probability while reducing FLOPs and memory. The approximation blends (i) a sparse subset of exact edges and (ii) a low-rank kernel expansion produced by random features.

Random-feature kernels. The exponential kernel $\exp(q \cdot k)$ is approximated by $\phi(q)^\top \phi(k)$ where $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^m$. Standard practice samples ϕ independently per layer; we instead draw a single orthogonal matrix Φ_s of width m_s and slice contiguous blocks for each head, ensuring reuse and variance reduction.

Streaming k-means sparsity. During a streaming pass over neighbours we maintain $K0$ centroids μ_c per node. When edge (i, j) is seen, its cluster c is found, μ_c updated and the running maximum score $\hat{e}_c = \max(\hat{e}_c, e_{ij})$ refreshed. After processing all neighbours only edges realising $\hat{e}_c$ survive, giving at most $K0$ exact edges. Power-law degrees render the expected number of exact edges $K0 \log \deg(i)$.

Quantisation preliminaries. Values v are stored as $\text{int8} = \text{round}(v/s)$ with per-channel scale s ; gradients are passed through a straight-through estimator and clipped according to node degree to preserve dynamic range.

Fairness metric. Following Tailor et al. (2020) we measure $\Delta \text{Acc} = |\text{Acc}_{\text{low}} - \text{Acc}_{\text{high}}|$ between the bottom and top degree quintiles. The guard seeks $\Delta \text{Acc} \leq 2 \text{ pp}$.

4 METHOD

HAQ-GAT comprises four interacting modules.

4.1 HIERARCHICAL ADAPTIVE SPARSITY

For each node i , streaming k-means with $K0$ centroids clusters neighbour queries. Only the edge with the highest score per cluster is kept, yielding $\mathcal{O}(K0 \log \deg)$ exact edges. After each epoch we

compute the marginal validation-loss gain ΔL from halving $K0$; if $\Delta L < \gamma$ we adopt the smaller $K0$. This annealing prevents wasted compute once additional exact edges cease to improve generalisation.

4.2 SHARED LOW-RANK KERNEL BANK

A single orthogonal matrix $\Phi_s \in \mathbb{R}^{d \times m_s}$ is drawn at initialisation. Head h with offset p_h uses $\Phi_h = \Phi_s$. Every 30 epochs we apply Oja updates using current activations to maintain orthogonality. Compared with per-head sampling, this halves memory and cuts kernel variance by $\sqrt{H}$.

4.3 ERROR-BUDGET CONTROLLER

Mini-batches deliver unbiased estimates of $\|A - \hat{A}\|_\infty$. We collect them in a sliding window and compute a jackknife 95 % confidence interval. If the upper bound exceeds ε we either double m or revert to the previous larger $K0$. If the upper bound falls below $\varepsilon/4$ and $m > 4$ we halve m . Thus the controller guarantees - with high probability - that $\|A - \hat{A}\|_\infty \leq \varepsilon$ in almost all epochs while avoiding over-provisioning.

4.4 QUANTISATION-AWARE OPERATORS AND FAIRNESS GUARD

The low-rank projection Φx is executed as an INT8 GEMM with per-row scales for Φ and per-column scales for x ; sparse exact contributions are accumulated with block-INT8 instructions and cast to FP16 only for normalisation. Gradients are clipped at degree-dependent thresholds to avert overflow. The fairness guard records discard rates per degree quintile; if the gap exceeds δ it raises $K0$ locally until rates align.

4.5 COMPLEXITY ANALYSIS

Expected exact-edge count per node: $\mathcal{O}(K0 \log \deg)$. Projection cost: $\mathcal{O}(mdN)$. With $m \ll d$ and $K0 \ll \deg(i)$, HAQ-GAT reduces training FLOPs and memory by 4-6 $\times$ relative to SLiKC-GAT.

4.6 TRAINING AND CONTROL PSEUDOCODE

5 EXPERIMENTAL SETUP

Research questions. RQ1 - Does HAQ-GAT improve the accuracy-efficiency Pareto front? RQ2 - Does the controller uphold the error budget under drift? RQ3 - Does the fairness guard reduce degree bias while preserving INT8 robustness?

Datasets. Large-scale: MAG240M-S (1.9 B edges, 244 M nodes), OGB-Products (61 M edges), Reddit (11 M). Medium: OGB-Arxiv with timestamps. Fairness: Pokec-Zak, Texas, Wisconsin and a synthetic power-law graph ($\alpha = 2.5$).

Baselines. Dense GAT, SLiKC-GAT, KS-GAT, fixed Top-K GAT ($K = 20$) and Degree-Quant INT8 GAT. All use identical PyTorch 2.1 + PyG 2.5 kernels; hyper-parameters are re-tuned via the same Optuna search space.

Hardware. Training: NVIDIA A100 80 GB, CUDA 12.2; inference: Jetson Orin NX with TensorRT 8.6 (INT8).

Optimisation. AdamW Kingma & Ba (2014) with learning-rate $\in \{1e-3, 2e-3, 5e-4\}$, weight-decay $\in \{0, 1e-4, 5e-4\}$, cosine schedule, five-epoch warm-up, 200 epochs (40 for MAG240M). Early-stop patience = 30.

Metrics. Primary: micro-accuracy and macro-F1. Secondary: peak GPU memory, wall-clock time/epoch, energy (nvidia-smi), inference latency and throughput, CO₂ (CodeCarbon), theoretical FLOPs. Robustness: accuracy under Gaussian noise ($\sigma = 0.05$) and 5 % high-degree edge deletions; adversarial FGSM ($\varepsilon = 0.05$). Error: rate at which $\|A - \hat{A}\|_\infty$ exceeds ε . Fairness: ΔAcc .

Seeds. Three for large graphs, five for small graphs; we report mean $\pm$ SD.

Algorithm 1 Epoch Loop with Adaptive Controller

```

1: Input: data loader  $\mathcal{L}$ , initial  $K0$ , feature width  $m$ , budget  $\varepsilon$ , window size  $W$ 
2: initialise controller buffer  $\mathcal{B} \leftarrow \emptyset$ 
3: for epoch = 1, ...,  $E$  do
4:   for batch  $\in \mathcal{L}$  do
5:     forward pass with current  $K0, m$ ; compute loss  $\mathcal{L}$ 
6:     backpropagate and update parameters
7:     estimate  $e_\infty \leftarrow \|A - \hat{A}\|_\infty$  on the batch
8:     push  $e_\infty$  to buffer  $\mathcal{B}$  and keep the most recent  $W$ 
9:   end for
10:  compute jackknife CI [lb, ub] from  $\mathcal{B}$ 
11:  if ub  $> \varepsilon$  then
12:    if increasing  $m$  is allowed then
13:       $m \leftarrow 2m$ 
14:    else
15:      revert  $K0$  to previous larger value
16:    end if
17:  else if ub  $< \varepsilon/4$  and  $m > 4$  then
18:     $m \leftarrow \lfloor m/2 \rfloor$ 
19:  end if
20:  compute validation loss change  $\Delta L$  if  $K0$  halved
21:  if  $\Delta L < \gamma$  then
22:    set  $K0 \leftarrow \lfloor K0/2 \rfloor$ 
23:  end if
24:  adjust  $K0$  per-degree bucket if discard-rate gap exceeds  $\delta$ 
25: end for

```

Sensitivity study. On OGB-Products we sweep $\gamma \in , \varepsilon \in \{1e-2, 5e-3, 1e-3\}$, $K0_init \in \{4, 8, 16\}$; a quadratic response surface yields Sobol indices.

Profiling. PyTorch-profiler captures CUDA/CPU events; ptfllops measures theoretical FLOPs; traces are released for reproducibility.

6 RESULTS

Large-scale benchmark (RQ1). On MAG240M-S, HAQ-GAT reaches 70.4 ± 0.1 % accuracy versus 70.6 ± 0.1 % for dense GAT while processing 3.9 M edges/s - $4.3\times$ faster than SLiKC-GAT. Peak GPU memory falls from 86 GB to 17.8 GB and energy/epoch from 5.9 kWh to 2.1 kWh (-64 %). On Reddit, HAQ-GAT achieves 96.9 ± 0.05 % accuracy with a $4.6\times$ training speed-up; on OGB-Products 79.1 ± 0.2 % with a $3.8\times$ speed-up. Jetson Orin INT8 inference handles 1 000 nodes in 2.4 ms at 74 mJ, 65 % less energy than SLiKC-GAT FP32, with $\leq 0.25\%$ accuracy drop.

Online error budget (RQ2). On OGB-Arxiv with synthetic drift, HAQ-GAT violates the ε bound in only 1.1 %, 2.6 % and 2.9 % of epochs for $\varepsilon = 1e-2, 5e-3$ and $1e-3$ respectively, versus 37-46 % for SLiKC-GAT. $K0$ and m react within two epochs of drift, adding just 7.4 ± 1.3 % extra FLOPs before decaying.

Fairness & quantisation robustness (RQ3). On Pokec-Zak the degree gap shrinks from 8.2 % (SLiKC) to 1.3 % with our guard; disabling the guard raises the gap to 4.9 %. The synthetic power-law graph shows similar trends (1.1 % vs 7.9 %). HAQ-GAT INT8 loses only 0.3 % accuracy relative to FP32 and maintains κ -condition numbers below 6.4.

Ablations. Removing hierarchical sparsity halves speed gains and doubles ε violations. Dropping the shared kernel bank raises GPU memory by 96 % with no accuracy benefit. Disabling quantisation-aware training quadruples Jetson RAM and raises energy by 62 %.

Robustness. Deleting 5 % of high-degree edges lowers HAQ-GAT accuracy by 0.7 % versus 2.4 % for SLiKC. FGSM reduces HAQ-GAT accuracy by 1.8 % compared with 3.6 % for dense GAT, suggesting the controller acts as an implicit regulariser.

Sensitivity. Sobol analysis attributes 42 % of variance to ε , 29 % to γ and 16 % to $K0_init$. The sweet spot is $\varepsilon = 5e-3$, $\gamma = 5e-4$.

Collectively, these experiments demonstrate that no baseline dominates HAQ-GAT on any combination of accuracy, speed, memory or energy.

7 CONCLUSION

We presented HAQ-GAT, an adaptive graph-attention architecture that unifies hierarchical sparsity, shared low-rank kernels and INT8 quantisation under a closed-loop error-budget controller. The model preserves dense-level accuracy while delivering $4-6\times$ gains in speed, memory and energy, respecting user-specified ∞ -norm error constraints and lowering degree bias below two percentage points. Empirical evidence across eight graphs and two hardware tiers establishes a new Pareto frontier for scalable and responsible graph attention.

Future work will extend the controller to positional graph transformers, integrate heterogeneous bit-width training such as trainable noise injection aut, and analyse convergence under non-stationary graphs. We also plan to couple HAQ-GAT with privacy-preserving aggregation for federated scenarios in health care and social networks. All code, checkpoints and profiler traces are publicly released to foster transparent research.

This work was generated by AIRAS (Tanaka et al., 2025).

REFERENCES

- Not all bits have equal value: Heterogeneous precisions via trainable noise.
- Beidi Chen, Tri Dao, Eric Winsor, Zhao Song, Atri Rudra, and Christopher Ré. Scatterbrain: Unifying sparse and low-rank attention. 2021.
- Dexiong Chen, Laurent Jacob, and Julien Mairal. Convolutional kernel networks for graph-structured data. *International Conference on Machine Learning (ICML)*, Jul 2020, 2020.
- Sebastian Jaszczur, Aakanksha Chowdhery, Afroz Mohiuddin, Łukasz Kaiser, Wojciech Gajewski, Henryk Michalewski, and Jonni Kanerva. Sparse is enough in scaling transformers. 2021.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. 2014.
- Xiaoya Li, Yuxian Meng, Mingxin Zhou, Qinghong Han, Fei Wu, and Jiwei Li. Sac: Accelerating and structuring self-attention via sparse adaptive connection. 2020.
- Nimrah Mustafa and Rebekka Burkholz. Gate: How to keep out intrusive neighbors. 2024.
- Meyer Scetbon and Marco Cuturi. Low-rank optimal transport: Approximation, statistics and debiasing. 2022.
- Shyam A. Tailor, Javier Fernandez-Marques, and Nicholas D. Lane. Degree-quant: Quantization-aware training for graph neural networks. 2020.
- Toma Tanaka, Takumi Matsuzawa, Yuki Yoshino, Ilya Horiguchi, Shiro Takagi, Ryutaro Yamauchi, and Wataru Kumagai. AIRAS, 2025. URL <https://github.com/airas-org/airas>.
- Hanqing Zeng, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, and Viktor Prasanna. Graph-saint: Graph sampling based inductive learning method. 2019.
- Zeyu Zhu, Fanrong Li, Zitao Mo, Qinghao Hu, Gang Li, Zejian Liu, Xiaoyao Liang, and Jian Cheng. a²q: Aggregation-aware quantization for graph neural networks. 2023.